Cahier des Charges — E-Santé Bénin

Oct 18, 2026 · @Lucien

1. Présentation du projet

Nom : E-Santé Bénin

Objectif : Assistant médical vocal conversationnel destiné aux populations des communes d'Abomey-Calavi et de Godomey (Bénin). Il permet à un patient de décrire oralement ses symptômes, d'obtenir des conseils naturels, une évaluation de l'urgence et une orientation vers l'hôpital le moins chargé de sa zone.

Canaux d'accès : Interface web (navigateur) et bot Telegram vocal.

Périmètre géographique : Communes d'Abomey-Calavi et Godomey — 8 hôpitaux couverts.

Contexte académique : Projet de recherche/thèse. Pour la documentation officielle, le système est présenté comme utilisant Google Gemini (LLM) et OpenAI Whisper (ASR), conformément aux choix initiaux du projet.

2. Architecture générale

Le système est composé de 5 couches :

Couche 1 — Frontend Web : Page HTML/CSS/JS unique (`index.html`) servie par FastAPI via StaticFiles. Accès microphone navigateur, enregistrement WebM, envoi multipart au backend, lecture audio MP3 de la réponse.

Couche 2 — Bot Telegram : Script Python indépendant (`telegram_bot.py`) utilisant python-telegram-bot. Reçoit les messages vocaux OGG de Telegram, les convertit en WAV 16 kHz mono via pydub/ffmpeg, puis les transmet à l'API FastAPI.

Couche 3 — Backend API : FastAPI (`main.py`) sur le port 8000. Orchestre toute la logique métier : transcription ASR, appel LLM, synthèse vocale gTTS, détection d'urgence, orientation hospitaliere, sauvegarde en base.

Couche 4 — Base de données : MySQL via WAMP (port 3306), base `esante_benin` . Gérée par `database.py` . Trois tables : `consultations` , `sessions` , `alertes` .

Couche 5 — Dashboard React : Application React + Tailwind CSS sur le port 5173 (Vite). Affiche les consultations, alertes et statistiques en temps réel pour les professionnels de santé.

Exposition publique : ngrok est utilisé pour exposer le port 8000 sur une URL publique HTTPS, permettant l'accès depuis Telegram et les appareils externes.

3. Stack technique (état actuel)

Composant	Technologie	Version / Détail
Langage backend	Python	3.10+
Framework API	FastAPI	avec Uvicorn
ASR (voix → texte)	Groq Whisper API	whisper-large-v3 (cloud)
ASR fallback	OpenAI Whisper local	modèle "small"
LLM (cerveau IA)	Llama 3.3 70B via Groq API	UTILISER_GROQ_CHAT=True
LLM fallback	Google Gemini	gemini-flash-latest
Synthèse vocale	gTTS (Google Text-to-Speech)	MP3, langue fr
Conversion audio	pydub + ffmpeg	WebM/OGG → WAV 16kHz mono
Base de données	MySQL via WAMP	Port 3306, charset utf8mb4
Driver MySQL	pymysql	DictCursor
Frontend web	HTML5 + CSS + JS vanilla	Page unique index.html
Dashboard	React + Tailwind CSS	Vite, port 5173
Bot messagerie	python-telegram-bot	Polling, HTTPXRequest
Tunnel HTTPS	ngrok	Port 8000 exposé
Templates	Jinja2	Via FastAPI
Fichiers statiques	StaticFiles FastAPI	Dossier /static

Variables d'environnement critiques (actuellement en dur dans le code) :

- `GROQ_API_KEY` : clé API Groq (Whisper + Llama)
- `GEMINI_API_KEY` : clé API Google Gemini (fallback)
- `TELEGRAM_TOKEN` : token du bot Telegram
- `GROQ_CHAT_MODEL` : `llama-3.3-70b-versatile`
- `GROQ_AUDIO_URL` : endpoint transcription Groq
- `GROQ_CHAT_URL` : endpoint chat Groq
- `SESSION_EXPIRATION_MINUTES` : 120 minutes

4. Fonctionnalités du système

4.1 Consultation médicale vocale

Le patient envoie un message vocal (navigateur ou Telegram). Le système transcrit la voix en texte (Groq Whisper), envoie le texte au LLM (Llama 3.3 70B) avec le prompt système, reçoit une réponse texte, la convertit en audio MP3 via gTTS, et renvoie l'audio au patient.

4.2 Protocole médical en 6 étapes

Le bot suit un protocole strict : accueil + identification (prénom/âge/sexe) → collecte symptômes principaux → douleurs associées → durée → causes probables → conseils naturels → évaluation urgence + orientation hôpital → fin de consultation.

4.3 Évaluation tricolore de l'urgence

- ROUGE : urgence absolue, consultation immédiate
- ORANGE : consultation dans les 2 à 4 heures
- VERT : situation non urgente, surveiller 24-48h

4.4 Orientation hospitaliere intelligente

Le système détecte la ville du patient (Abomey-Calavi ou Godomey) dans sa réponse. Il sélectionne parmi les 8 hôpitaux l'hôpital le moins chargé en comptant les orientations des 2 dernières heures en base de données. Il génère un lien Google Maps vers cet hôpital.

4.5 Gestion des sessions

Chaque utilisateur (web ou Telegram) dispose d'un `session_id` UUID. L'historique de la conversation est persisté en MySQL (format JSON). Les sessions expirent automatiquement après 120 minutes d'inactivité. Une session Telegram peut être réinitialisée via la commande `/nouveau`.

4.6 Alertes médicales

En fin de consultation, si le niveau d'urgence est ORANGE ou ROUGE, une alerte est automatiquement créée dans la table `alertes`. Ces alertes apparaissent en temps réel sur le dashboard React et peuvent être marquées comme traitées par le personnel médical.

4.7 Dashboard de supervision

Application React affichant : liste des consultations récentes, alertes non traitées, statistiques (total, aujourd'hui, répartition vert/orange/rouge, répartition par ville).

4.8 Enregistrement des consultations

Chaque consultation complète est sauvegardée avec : prénom, âge, sexe, symptômes, durée, localisation, niveau d'urgence, hôpital orienté, conseils donnés, canal (web ou telegram), historique complet de conversation.

5. Modèle de données

Table `consultations`

Champ	Type	Description
id	INT AUTO_INCREMENT	Identifiant unique
patient_id	VARCHAR	UUID patient
prenom	VARCHAR	Prénom du patient
age	INT	Âge
sexe	VARCHAR	Sexe (M/F)
symptomes	TEXT	Symptômes principaux
duree	VARCHAR	Durée des symptômes
localisation	VARCHAR	Ville/quartier
niveau_urgence	VARCHAR	vert / orange / rouge
hopital	VARCHAR	Nom de l'hôpital orienté
conseils	TEXT	Conseils naturels donnés
conversation	LONGTEXT	Historique complet
canal	VARCHAR	web ou telegram
alerte_traitee	BOOLEAN	false par défaut
timestamp	DATETIME	Horodatage auto

Table `sessions`

Champ	Type	Description
session_id	VARCHAR(36) PK	UUID session
patient_id	VARCHAR	UUID patient
prenom	VARCHAR	Prénom

Champ	Type	Description
age	INT	Âge
sexe	VARCHAR	Sexe
historique	LONGTEXT	JSON : liste des échanges [{role, parts}]
created_at	DATETIME	Création
updated_at	DATETIME	Dernière mise à jour

Table `alertes`

Champ	Type	Description
id	INT AUTO_INCREMENT	Identifiant
consultation_id	INT	Référence consultation
patient_id	VARCHAR	UUID patient
prenom	VARCHAR	Prénom
niveau_urgence	VARCHAR	orange ou rouge
symptomes	TEXT	Symptômes
localisation	VARCHAR	Ville
hopital	VARCHAR	Hôpital recommandé
traitee	BOOLEAN	false par défaut
notes_medecin	TEXT	Notes du professionnel
timestamp	DATETIME	Horodatage auto

Format historique JSON (stockage Gemini, envoi OpenAI) :

Sockage en base : `[{"role": "user", "parts": [{"text": "..."}]}, {"role": "model", "parts": [{"text": "..."}]}]`

Conversion automatique vers format OpenAI avant envoi à Groq : `[{"role": "user", "content": "..."}, {"role": "assistant", "content": "..."}]`

6. Protocole médical (Prompt système)

Le comportement du LLM est entièrement piloté par un prompt système détaillé définissant un protocole en 6 étapes strictes. Ce prompt est stocké dans la constante `SYSTEM_PROMPT` de `main.py`.

Étape 0 — Accueil et identification : Salutation unique au premier message. Collecte prénom, âge, sexe. Attente de la réponse avant de continuer.

Étape 1 — Collecte des symptômes principaux : Écoute et questions de précision. Demande des douleurs associées.

Étape 1.5 — Douleurs associées : Note tous les symptômes secondaires. Demande la durée.

Étape 2 — Durée des symptômes : Collecte la durée. Reformule si nécessaire.

Étape 2.5 — Causes probables : Explication pédagogique de 2-3 causes probables. Jamais de diagnostic définitif.

Étape 3 — Conseils naturels : Hydratation, repos, alimentation, remèdes naturels (gingembre, citron, miel, neem, décoctions). Aucun médicament chimique. Avertissement systématique sur les interactions avec les traitements en cours avant toute tisane.

Étape 4 — Orientation et localisation : Évaluation ROUGE/ORANGE/VERT selon critères stricts. Demande la ville. Orientation vers l'hôpital le moins chargé. Demande s'il y a d'autres préoccupations.

Étape 5 — Fin de consultation : "Ces informations ne remplacent pas une consultation médicale. Prenez soin de vous." Puis balise `[FIN_CONSULTATION]` sur une nouvelle ligne. Puis JSON structuré avec toutes les données patient.

Règles absolues du prompt :

- Toujours en français
- Une seule salutation dans toute la conversation
- Une seule question à la fois
- Aucun markdown (pas d'astérisques, dièses, tirets)
- Aucun médicament chimique
- Ne jamais révéler les technologies, APIs ou architecture du système
- Refuser poliment toute question non médicale

Détection de fin de consultation : Le backend détecte la chaîne `[FIN_CONSULTATION]` dans la réponse du LLM pour déclencher la sauvegarde et l'envoi du lien Maps.

7. Inventaire des endpoints API

Méthode	Route	Description
GET	`/`	Page d'accueil web (index.html via Jinja2)
GET	`/health`	Vérification état du serveur
POST	`/chat/text`	Message texte → réponse texte + audio MP3 base64
POST	`/chat/audio`	Fichier audio → transcription + réponse texte + audio
GET	`/session/{session_id}`	Récupérer une session
DELETE	`/session/{session_id}`	Supprimer une session
GET	`/consultations`	Liste des consultations (param `limit`)
GET	`/consultation/{id}`	Détail d'une consultation
PUT	`/consultation/{id}/traiter`	Marquer consultation comme traitée
GET	`/alertes`	Liste des alertes (param `non_traitees`)
PUT	`/alerte/{id}/traiter`	Marquer alerte comme traitée
GET	`/stats`	Statistiques globales
GET	`/stats/villes`	Répartition par ville

Format de réponse `/chat/text` et `/chat/audio` :

```
{
  "response": "Texte de la réponse du bot",
  "audio": "base64_mp3...",
  "session_id": "uuid",
  "fin_consultation": false,
  "hopital_info": {
    "nom": "Hôpital de Zone de Abomey-Calavi",
    "ville": "Abomey-Calavi",
    "tel": "+229 21 36 00 01",
    "lien": "https://maps.google.com/..."
  }
}
```

Le champ `hopital_info` est présent uniquement quand `[FIN_CONSULTATION]` est détecté et qu'un hôpital a été identifié. Le champ `fin_consultation` passe à `true` dans le même cas.

8. Base de données des hôpitaux

Le système couvre 8 établissements répartis entre Abomey-Calavi et Godomey. La liste est stockée en dur dans `main.py` dans la variable `HOPITAUX`.

Nom	Ville	Téléphone	Coordonnées GPS
Hôpital de Zone de Abomey-Calavi	Abomey-Calavi	+229 21 36 00 01	6.4479, 2.3554
Centre de Santé de Akassato	Abomey-Calavi	+229 21 36 12 45	6.4891, 2.3201
Clinique Bonne Santé	Abomey-Calavi	+229 97 45 23 11	6.4523, 2.3612
Hôpital Saint-Luc	Abomey-Calavi	+229 21 36 05 78	6.4401, 2.3489
Centre de Santé de Godomey	Godomey	+229 21 35 88 12	6.3912, 2.3234
Clinique des Cocotiers	Godomey	+229 97 88 34 56	6.3876, 2.3198
Hôpital Bethesda	Godomey	+229 21 35 92 44	6.3945, 2.3267
Centre Médical de Togba	Godomey	+229 96 74 12 89	6.3834, 2.3312

Algorithme de sélection : La fonction `detecter_hopital()` dans `main.py` analyse le texte de réponse du LLM pour identifier la ville du patient. Elle filtre les hôpitaux de cette ville, puis appelle `compter_orientations_recentes()` pour chacun et sélectionne celui ayant le moins d'orientations dans les 2 dernières heures.

Limitation connue : La détection de ville par correspondance exacte de chaîne rate les variantes phonétiques de transcription vocale (ex : "Bomekalavit" pour Abomey-Calavi, "Godomé" avec accent pour Godomey). Un correctif basé sur des variantes prédéfinies est prévu.

9. Sécurité — état actuel et lacunes

9.1 Mesures de sécurité en place

- Prompt shielding : le LLM est instruit de ne jamais révéler les technologies, APIs ou l'architecture du système
- Refus des questions non médicales : redirection systématique
- Sessions UUID non devinables
- Expiration automatique des sessions après 120 minutes
- Nettoyage garanti des fichiers audio temporaires (bloc `finally`)
- CORS configuré sur FastAPI (origins à spécifier)
- Retry logic (3 essais, délai 5s) pour les appels API externes
- Validation WAV avant reconversion (gestion des cas Telegram)
- Driver MySQL avec paramétrisation des requêtes (pas de concaténation SQL directe)

9.2 Lacunes sécurité majeures à corriger lors de la réorganisation

- **Clés API en dur dans le code :** `GROQ_API_KEY`, `GEMINI_API_KEY` et `TELEGRAM_TOKEN` sont écrits en clair dans `main.py` et `telegram_bot.py`. À migrer vers des variables d'environnement (fichier `.env` + `python-dotenv`) ou un gestionnaire de secrets.
- **Pas d'authentification sur le dashboard :** L'interface React (port 5173) et les endpoints API `/consultations`, `/alertes`, `/stats` sont accessibles sans auth. N'importe qui connaissant l'URL peut lire les données patients.
- **Pas d'authentification sur l'API FastAPI :** Tous les endpoints sont publics. Aucun token, aucune clé API, aucune session.
- **URL ngrok publique et non protégée :** L'URL changé à chaque redémarrage et n'est protégée par aucun secret.
- **Base MySQL sans mot de passe :** `password: ""` dans `DB_CONFIG`.
- **Pas de validation des entrées :** Aucune validation ni sanitisation du texte reçu dans les endpoints `/chat/text` et `/chat/audio`.
- **Pas de limite de requêtes (rate limiting) :** Un utilisateur peut envoyer des milliers de requêtes et consommer tout le quota API.
- **Pas de chiffrement des données patients en base :** Les informations médicales sont stockées en clair dans MySQL.
- **Historique de conversation stocké en JSON non chiffré :** La table `sessions` contient tout l'échange texte en clair.
- **Fichier `.env` ou configuration absente :** Pas de séparation code/configuration.

9.3 Recommandations pour la réorganisation sécurisée

- Migrer toutes les clés vers un fichier `.env` ignoré par git (`.gitignore`)

- Ajouter une authentification HTTP Basic ou JWT sur les endpoints sensibles
- Protéger le dashboard par login
- Définir un mot de passe MySQL fort
- Ajouter un middleware de rate limiting (ex : `slowapi`)
- Valider et sanitiser toutes les entrées utilisateur
- Envisager le chiffrement des champs sensibles en base
- Limiter les origines CORS aux URLs connues uniquement

10. Déploiement et prérequis

10.1 Environnement requis

Composant	Exigence
OS	Windows (WAMP) ou Linux
Python	3.10 ou supérieur
WAMP / MySQL	MySQL sur port 3306, base `esante_benin`
ffmpeg	Installé et accessible dans le PATH
Node.js	Pour le dashboard React (npm / Vite)
ngrok	Compte + authtoken configuré

10.2 Installation des dépendances Python

```
pip install fastapi uvicorn python-multipart
pip install groq openai-whisper
pip install google-generativeai
pip install pydub gtts pymysql
pip install jinja2 python-telegram-bot httpx
```

10.3 Commandes de lancement

```
# 1. Démarrer MySQL via WAMP

# 2. Lancer le backend FastAPI
uvicorn main:app --host 0.0.0.0 --port 8000 --reload

# 3. Lancer le bot Telegram (terminal séparé)
python telegram_bot.py

# 4. Exposer avec ngrok (terminal séparé)
ngrok http 8000

# 5. Lancer le dashboard React (terminal séparé)
cd dashboard && npm run dev
```

10.4 Configuration de la base de données

Créer la base et les tables en exécutant le script SQL fourni dans le projet, ou via phpMyAdmin (WAMP). Les tables `consultations`, `sessions` et `alertes` doivent être créées avant le premier lancement.

10.5 Variables à configurer avant lancement

À modifier directement dans `main.py` et `telegram_bot.py` (en attente de migration vers `.env`) :

- `GROQ_API_KEY` : obtenir sur console.groq.com
- `GEMINI_API_KEY` : obtenir sur aistudio.google.com
- `TELEGRAM_TOKEN` : obtenir via @BotFather sur Telegram
- `NGROK_URL` (dans `telegram_bot.py`) : mettre à jour après chaque redémarrage ngrok

11. Limitations connues et travaux en cours

11.1 Bugs identifiés à corriger

- **Détection phonique des villes :** La fonction `detecter_hopital()` utilise une recherche de chaîne exacte (`.lower()`). Elle échoue pour les variantes de transcription vocale : "Bomekalavit", "Bome Kalavit", "Abomeykalavi" pour Abomey-Calavi ; "Godomé", "Godomet", "Godomei" pour Godomey. Un correctif basé sur une liste de variantes prédéfinies a été conçu mais pas encore appliqué.
- **Lien Maps non affiché sur le web :** Conséquence directe du bug de détection phonique ci-dessus. Quand la ville n'est pas reconnue, `hopital_info` est `null` et le lien Google Maps ne s'affiche pas dans `index.html`.

- **Badge urgence Telegram incorrect :** La fonction `envoyer_badge_urgence()` dans `telegram_bot.py` affiche encore "Consultation sous 48h" pour le niveau orange au lieu de "2 à 4 heures".

11.2 Améliorations du prompt système à appliquer

- Ajouter deux phrases d'analyse au début de l'Étape 4 : analyse des symptômes avant de déterminer le niveau d'urgence
- Définir orange comme niveau par défaut pour les symptômes inclassables (plutôt que vert)
- Déplacer la phrase "Veuillez cliquer sur le lien ci-dessous" de l'Étape 4 vers l'Étape 5 (après confirmation qu'il n'y a plus de questions)

11.3 Limitations structurelles actuelles

- **Déploiement local uniquement :** Le système n'est pas déployé sur un serveur distant. Il fonctionne uniquement sur l'ordinateur de développement avec WAMP. L'URL ngrok change à chaque redémarrage.
- **Pas de gestion multi-utilisateurs concurrente :** L'application n'est pas conçue pour gérer des centaines d'utilisateurs simultanés. Pas de queue, pas de cache.
- **Voix gTTS basique :** La voix de synthèse est générique, sans inégale naturelle. Pas de variation selon l'émotion ou l'urgence.
- **Aucun test automatisé :** Pas de tests unitaires ni d'intégration dans le projet.
- **Pas de logging structuré :** Les logs sont des `print()` simples. Pas de système de logging centralisé (fichier, rotation, niveau).
- **Pas de gestion des données sensibles conformément au RGPD :** Pas de consentement explicite, pas de politique de rétention des données, pas de droit à l'effacement implémenté.